

PROOF PUNCHLINES – say only the load-bearing line

- **BM safe:** the stored shift (gs / R_k / mp) is the RIGHTMOST/LONGEST match, so a smaller shift would need a closer match that does not exist $\Rightarrow$ skip nothing.
- $\sqrt{1} \bmod p$: $x^2=1 \Rightarrow (x-1)(x+1) \equiv 0 \bmod p \Rightarrow p \mid (x-1)$ or $p \mid (x+1) \Rightarrow x = \pm 1$.
- **Fib max-degree:** at link x had $i-1$ kids & degrees must be EQUAL, so $\deg(y_i) \geq i-1$; minus the ≤ 1 child it later lost $= i-2$; subtree size $\geq F_{k+2} \geq \varphi^k$.
- **Suffix tree:** unique $\$ \Rightarrow$ no suffix is a prefix of another $\Rightarrow$ exactly n leaves; every internal node branches $\geq 2 \Rightarrow \leq n-1$ internal nodes.
- **B-tree height** grows ONLY when the root splits (a new root is made from the median).

DON'T-SLIP – exact values you have gotten wrong

- Z border length: condition is $i + Z[i] = n+1$ (NOT n); border $= Z[i]$.
- B-tree height: $h \leq \log_t \frac{N+1}{2}$ and $h \geq \log_{2t}(N+1) - 1$ (mind the -1).
- Amortized = TOTAL/ n : k -bit counter total flips $= 2n \Rightarrow O(1)$ per op (not $O(n)$).
- BWT inversion: FOLLOW the LF chain from the $\$$ row; do not read L in row order.
- $LF(i) = C[L[i]] + Occ(L[i], i)$; $C[c] = \#\{\text{chars} < c\}$, $Occ =$ count of c in $L[1..i]$.

wk01 Z-algorithm $O(n)$

IDEA $Z[i]$ = length of the longest substring at i matching a prefix of S ; $Z[1] = n$.
 Window $[l, r]$, $k' = i - l + 1$: if $Z[k'] < r - i + 1$ copy; $>$ clamp to $r - i + 1$; = extend past r .
Exact match $S = \text{pat} \$ \text{txt}$: where $Z[i] = m \Rightarrow$ hit at $i - (m+1)$.
Period smallest $p \mid n$ with $Z[p+1] = n - p$, else n .
Border $i + Z[i] = n+1 \Rightarrow$ border len $Z[i]$.
 $Z^{suf} = \text{rev}(Z(\text{rev}S))$.

wk02 Boyer-Moore (scan R $\rightarrow$ L)

IDEA on mismatch at k , jump by $\max(\text{bad-char}, \text{good-suffix})$.
Bad char $= k - R_k(x)$. $R_k(x)$ = rightmost POSITION of x to the left of k (a position, not a count); 0 if none.
Good suffix $= m - gs[k+1]$; fallback matched-prefix $= m - mp$.
 ► Build gs: for each i set $gs[m - Z_i^{suf} + 1] = i$.
 gs stores the RIGHTMOST such re-occurrence $\Rightarrow$ shift is safe. Worst $O(n+m)$.

wk03 BWT

Build $L[i] = S[SA[i] - 1]$ ($SA[i] = 1 \Rightarrow \$$); $F = \text{sort}(L)$.
 $LF(i) = C[L[i]] + Occ(L[i], i)$
 ► **Invert:** follow LF from the $\$$ -row, collect each L , then REVERSE $= S$.
 ► **Backward search** pat R $\rightarrow$ L, $[sp, ep] = [1, n]$, per char c :
 $sp = C[c] + Occ(c, sp-1) + 1$
 $ep = C[c] + Occ(c, ep)$ (empty if $sp > ep$)
 $\# \text{hits} = ep - sp + 1$; positions $SA[sp..ep]$.

WORKED

$S = \text{banana}\$$
 $i : 1 2 3 4 5 6 7$
 $L : a n n b \$ a a$
 $F : \$ a a a b n n$
 $LF : 2 6 7 5 1 3 4$
 $\$$ -row=5; chain $5 > 1 > 2 > 6 > 3 > 7 > 4$
 read $L = \$, a, n, a, n, a, b$; reverse = $\text{banana}\$$

wk04 Ukkonen suffix tree $O(n)$

Each extension is ONE of 3 rules:
 1. leaf path: extend for free via global end.
 2. ends & next char absent $\Rightarrow$ new leaf (and SPLIT = new internal node if mid-edge).
 3. char already present $\Rightarrow$ SHOWSTOPPER: stop the phase, resume next.
Suffix link $\text{node}(ca) \rightarrow \text{node}(a)$.
 $O(n)$ from: links + skip/count + global end + rule-3 early stop.
Apps occ count = $\#$ leaves under the node ($O(n)$ DFS prep, $O(m)$ query); longest repeat = deepest internal node.

wk05 Compression

Elias- ω encode N : write min-binary of N ; recursively PREPEND $L = (\text{len} - 1)$ of the previous part, until a part has length 1; flip the MS-B 1 $\rightarrow$ 0 of every LENGTH part; the value N keeps its leading 1.
 ► **Decode:** read a part. MSB 0 $\Rightarrow$ it is a length: flip to 1, next part has len = val+1. MSB 1 $\Rightarrow$ this part is the final value.
LZ77 $\langle \text{off}, \text{len}, \text{ch} \rangle$: longest window match (overlap allowed) then 1 literal. Decode: copy len chars from off back, append ch.
Huffman merge the 2 smallest; codelen = depth; total bits = $\sum f \cdot \text{depth}$.

WORKED

LZ77 decode $\langle \text{off}, \text{len}, \text{ch} \rangle$:
 $\langle 0, 0, a \rangle \rightarrow a$
 $\langle 1, 4, b \rangle \rightarrow \text{aaaaab}$ (copy 4 @1-back, overlap)
 $\langle 2, 2, c \rangle \rightarrow \text{aaaaababc}$

Pick the right tool (strings)

- 1 pattern in 1 text: **Z** or **Boyer-Moore** (BM skips more in practice). $O(n+m)$.
- many patterns vs fixed text: preprocess into **suffix tree** (Ukkonen) or **BWT/FM-index**.
- count / locate occurrences: leaves under a suffix-tree node, or **BWT backward search**.
- find repeats / period: deepest internal node; Z-period rule.
- compress: **Huffman** (char freq) · **LZ77** (repeats) · **Elias- ω** (integers).

wk06 Number theory

Karatsuba $uv = 2^n P_1 + 2^{n/2}(P_3 - P_1 - P_2) +$

$P_1 = U_1 V_1, P_2 = U_0 V_0, P_3 = (U_1 + U_0)(V_1 + V_0).$
 $T(n) = 3T(\frac{n}{2}) + O(n) = O(n^{1.585}).$

Mod-exp repeated squaring: $O(\log b)$ multiplications.

► **Miller-Rabin**: write $n-1 = 2^s t$ (t ODD).

Build $a^t, a^{2t}, \dots, a^{2^{s-1}t}$ by squaring.

COMPOSITE if $a^{n-1} \neq 1$ OR a nontrivial $\sqrt{1}$ appears ($x^2=1$ with $x \neq \pm 1$). Witness = a base proving compositeness. Error $\leq 4^{-k}$.

wk07 Fibonacci heap

insert / merge lazy: splice trees into root list, update min. $O(1)^*$

► **extract-min**: remove min, promote its children to roots, then CONSOLIDATE: degree array, link equal-degree trees (smaller key becomes parent). $O(\log n)^*$

► **decrease-key**: cut node to root; CASCADING-CUT its parent (mark on its 1st lost child, cut on the 2nd). $O(1)^*$

$$\Phi = \# \text{trees} + 2 \cdot \# \text{marked}$$

(* = amortised; actual extract-min $O(n)$)

wk08 Amortized analysis

Aggregate worst-case total over a sequence, divided by n .

Accounting charge $\geq$ actual cost, credit never negative (stack: push 2, pop 0; queue from 2 stacks: enqueue 3 = 2 credit/elem, dequeue 1).

Potential $a_i = c_i + \Delta\Phi$, with $\Phi(D_0)=0$ and $\Phi \geq 0$ always.

binary counter $\Phi = \#1\text{-bits}$; dynamic array $\Phi = 2 \cdot \text{size} - \text{cap}$.

wk09 B-trees (min degree t)

keys per node $t-1..2t-1$; children $t..2t$; root ≥ 1 key.

$$h \leq \log_t \frac{N+1}{2}$$

$$h \geq \log_{2t}(N+1) - 1$$

► **split full child** ($2t-1$ keys): median $key[t]$ goes UP to parent; left gets $key[1..t-1]$, right gets $key[t+1..2t-1]$ (split the child pointers too). $O(t)$.

insert proactive: split any full node on the way down (height grows only at a root split).

delete replace an internal key with its predecessor; borrow if a sibling has $>t-1$ keys, else merge.

WORKED

$t=3$, full node = [a b c d e]

median c moves UP

left = [a b] right = [d e]

wk10 Simplex (LP)

Standard form max, all $\leq$ constraints, $x \geq 0$, add 1 slack per constraint.

1. **entering** = MOST POSITIVE objective-row coeff.

2. **leaving** = min ratio $\frac{\text{RHS}}{\text{+ve col entry}}$.

3. pivot on that cell; repeat.

OPTIMAL: all obj coeffs ≤ 0

UNBOUNDED: entering col all ≤ 0

WORKED

max $3x_1 + 2x_2$; $x_1 + x_2 \leq 4$; $x_1 + 3x_2 \leq 6$

$x_1 \ x_2 \ s_1 \ s_2 | \text{RHS}$

$s_1 \ 1 \ 1 \ 1 \ 0 | 4$

$s_2 \ 1 \ 3 \ 0 \ 1 | 6$

ob 3 2 0 0 | 0

enter x1; ratios 4,6 -> s1 leaves

x1 1 1 1 0 | 4

s2 0 2 -1 1 | 2

ob 0 -1 -3 0 | -12 all ≤ 0 OPT z=12

wk11 Hungarian (assignment)

Minimise:

1. row-reduce (subtract row min), then col-reduce.

2. cover ALL zeros with the MINIMUM number of lines.

3. if $\# \text{lines} = n \Rightarrow$ optimal: choose INDEPENDENT zeros (one per row & column, not the diagonal).

► else subtract min-uncovered from UNCOVERED, ADD it to DOUBLY-covered cells (once-covered unchanged); go to 2.

Maximise: replace $a_{ij} \rightarrow M - a_{ij}$ ($M = \max$), then minimise. Cost $O(n^3)$.

WORKED

C = 4 2 8 / 4 3 7 / 3 1 6

row+col reduce -> cover row2,col2 (2<3)

1 0 2

0 0 0

1 0 1

min uncov=1: -1 uncov, +1 at (2,2):

0 0 1

0 1 0

0 0 0 -> zeros (1,1)(2,3)(3,2)=12

Complexities (memorise)

Z / BM / Ukkonen build $O(n)$; exact search $O(n+m)$; suffix-tree occ query $O(m)$.

Karatsuba $O(n^{1.585})$; mod-exp $O(\log b)$; Miller-Rabin $O(k \log^3 n)$.

Fib heap ins/merge/dec-key $O(1)^*$, extract-min $O(\log n)^*$.

B-tree search/insert/delete $O(t \log_t N)$.

Hungarian $O(n^3)$; Simplex exp worst-case / fast in practice.